



TBWB

Angular Advanced Dynamic configuration



Peter Kassenaar
info@kassenaar.com



Dynamic Configuration

Elevating the “build once, deploy everywhere” mantra

“Build once, deploy everywhere”

- Developers dream: Building **one single /dist artefact**
- Decide **at runtime** per client/environment/container which API's, endpoints, flags, variables, etc. are active
- We have two options: **Static** and **Dynamic** building

Configuring application environments

You can define different named build configurations for your project, such as **development** and **staging**, with different defaults.

Each named configuration can have defaults for any of the options that apply to the various builder targets, such as **build**, **serve**, and **test**. The [Angular CLI](#) **build**, **serve**, and **test** commands can then replace files with appropriate versions for your intended target environment.

Traditional – Static building

- `environment.ts` (and `environment.prod.ts`, etc) is selected via Angular CLI file replacements in `angular.json` during build.
- Result: **different build per environment** (`dev|stage|prod` etc.).

*Verdict: **very common**, but **inconvenient with Docker/Kubernetes/multi-tenant**: you have to rebuild if only a URL or feature flag changes.*

(because of the static nature of `environment.ts`)

Alternative – Dynamic building

- **Dynamic (runtime)**: an unbundled `./assets/config.json` is retrieved via HTTP at startup.
- Result: **same build, different configuration** per deployment
- You mount/overwrite that JSON next to `index.html`.
- Using `APP_INITIALIZER`:
 - Angular offers an **init hook that allows you to enforce asynchronous setup** before the app is “ready” (bootstrap).
 - Initialiser returns a Promise/Observable, Angular waits for it.
 - Modern Angular variants have a helper `provideAppInitializer(...)` which is standalone-friendly.

provideAppInitializer

@angular/core

provideAppInitializer function stable



The provided function is injected at application startup and executed during app initialization. If the function returns a Promise or an Observable, initialization does not complete until the Promise is resolved or the Observable is completed.

APP_INITIALIZER

API

```
function provideAppInitializer(initializerFn: () => any): EnvironmentProviders;
```

Description

The provided function is injected at application startup and executed during app initialization. If the function returns a Promise or an Observable, initialization does not complete until the Promise is resolved or the Observable is completed.

<https://angular.dev/api/core/provideAppInitializer>

Good article on Dynamic Configuration

DEV

Find related posts...Powered by Algolia

**seanbh**
Posted on 15 jul 2023 • Originally published at [Medium](#) on 14 jul 2023

 2

How to Get Environment Specific Configuration at Runtime in Angular

[#initialization](#) [#environment](#) [#configuration](#) [#api](#)


2


0


1

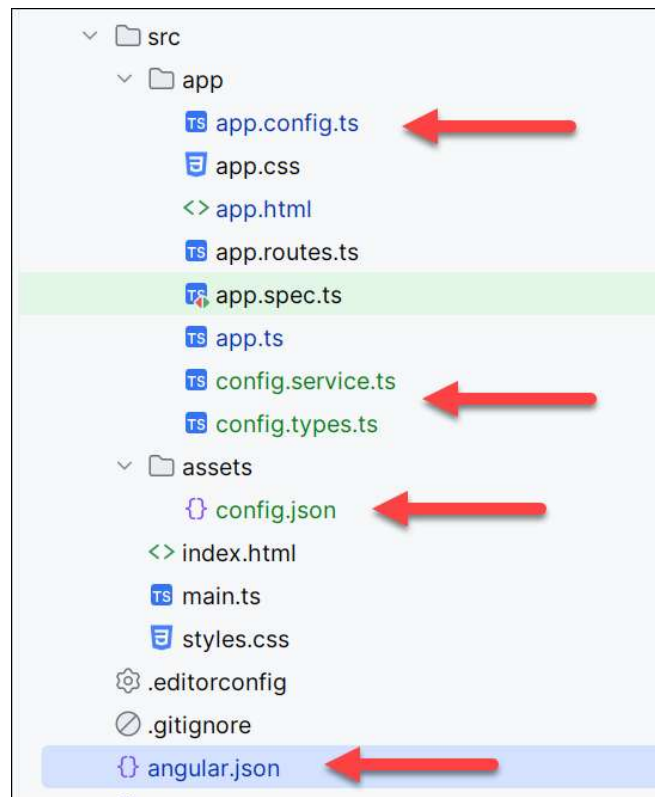


<https://dev.to/seanbh/how-to-get-environment-specific-configuration-at-runtime-in-angular-phb>

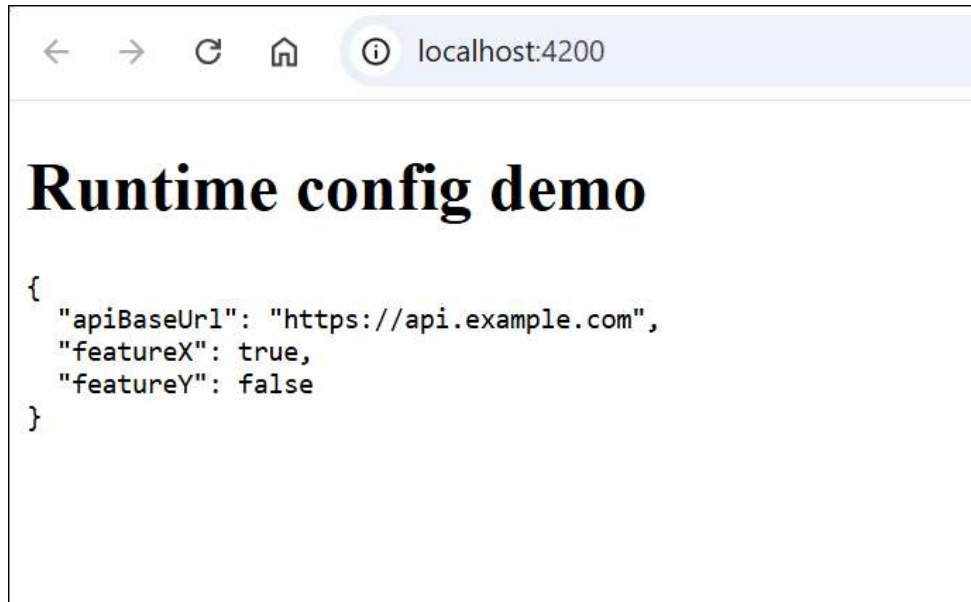
Minimal Example

- Requirements:
 - Modern Angular (preferably 17+)
 - Runtime config via `./assets/config.json`
 - Loading before bootstrapping the application

- Structure:



For now: just copy the contents of config file



File assets/config.json

```
{  
  "apiBaseUrl": "https://api.example.com",  
  "featureX": true,  
  "featureY": false  
}
```

Interface config.types.ts

```
export interface AppRuntimeConfig {  
  apiBaseUrl: string;  
  featureX: boolean;  
  featureY: boolean;  
}
```

Config.service.ts - responsible for loading the configuration and returning it to the app

```
import {inject, Injectable} from '@angular/core';
...

@Injectable({ providedIn: 'root' })
export class ConfigService {
  private _config!: AppRuntimeConfig;
  private http = inject(HttpClient);

  // getter for private variable
  get config(): AppRuntimeConfig {
    if (!this._config) {
      throw new Error('Config not loaded yet');
    }
    return this._config;
  }

  async load(): Promise<void> {
    const url = `assets/config.json?v=${Date.now()}`; // cache-bust
    this._config = await firstValueFrom(this.http.get<AppRuntimeConfig>(url));
  }
}
```

App.config.ts – provide configuration service

```
import {ApplicationConfig, ...} from '@angular/core';
import {ConfigService} from './config.service';
...

export const appConfig: ApplicationConfig = {
  providers: [
    ...
    // see separate config.service.ts for details on .load()
    provideAppInitializer(() => inject(ConfigService).load()),
  ]
};
```

App.ts | app.html

```
import {ConfigService} from './config.service';
import {AppRuntimeConfig} from './config.types';

...
export class App {
  private configService = inject(ConfigService);
  cfg: AppRuntimeConfig = this.configService.config;
}
```

```
<h1>Runtime config demo</h1>
<pre>{{ cfg | json }}</pre>
<router-outlet></router-outlet>
```

Runtime config demo

```
{
  "apiBaseUrl": "https://api.example.com",
  "featureX": true,
  "featureY": false
}
```

Usage of configuration parameters in component

- See example.component.ts|html|css in example application



Runtime config demo

```
{
  "apiBaseUrl": "https://api.example.com",
  "featureX": true,
  "featureY": false
}
```

Runtime config usage

API base URL: https://api.example.com

Dummy(!) Load data: Feature X from config file

(see what happens when you change FeatureX: true to false in assets/config.json)

